

[--- CODE CELL ---]

```
pip install --upgrade typing_extensions
```

[--- CODE CELL ---]

```
import torch
import matplotlib.pyplot as plt

# Reproducibility
torch.manual_seed(0)

# Parameters
num_samples = 100
true_w, true_b = 2.0, -1.0
noise_std = 0.7

# Features
X = torch.rand(num_samples)

# Noise
noise = torch.randn(num_samples) * noise_std

# Labels
y = true_w * X + true_b + noise

# Plot
plt.scatter(X.numpy(), y.numpy(), label="Synthetic data", alpha=0.7)
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

[--- CODE CELL ---]

```
import torch
import matplotlib.pyplot as plt

# Initial parameters
w = torch.tensor(0.0)
b = torch.tensor(0.0)

epochs = 50
lr = 0.05
losses = []

for epoch in range(epochs):
    for i in range(num_samples):
        xi = X[i]
        yi = y[i]

        # Prediction
        y_pred = w * xi + b

        # Loss
        loss = (y_pred - yi) ** 2
        losses.append(loss.item())

        # Gradients (manual)
        grad_w = 2 * (y_pred - yi) * xi
        grad_b = 2 * (y_pred - yi)

        # Parameter update
        w = w - lr * grad_w
        b = b - lr * grad_b

# Plot data + fitted line
plt.scatter(X.numpy(), y.numpy(), label="Synthetic data", alpha=0.7)
plt.plot(X.numpy(), (w * X + b).detach().numpy(), 'r', label="Regression line")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Online training data")
plt.legend()
plt.show()
```

```
print(f"Learned parameters: w={w.item():.3f}, b={b.item():.3f}")
```

[--- CODE CELL ---]

[--- CODE CELL ---]